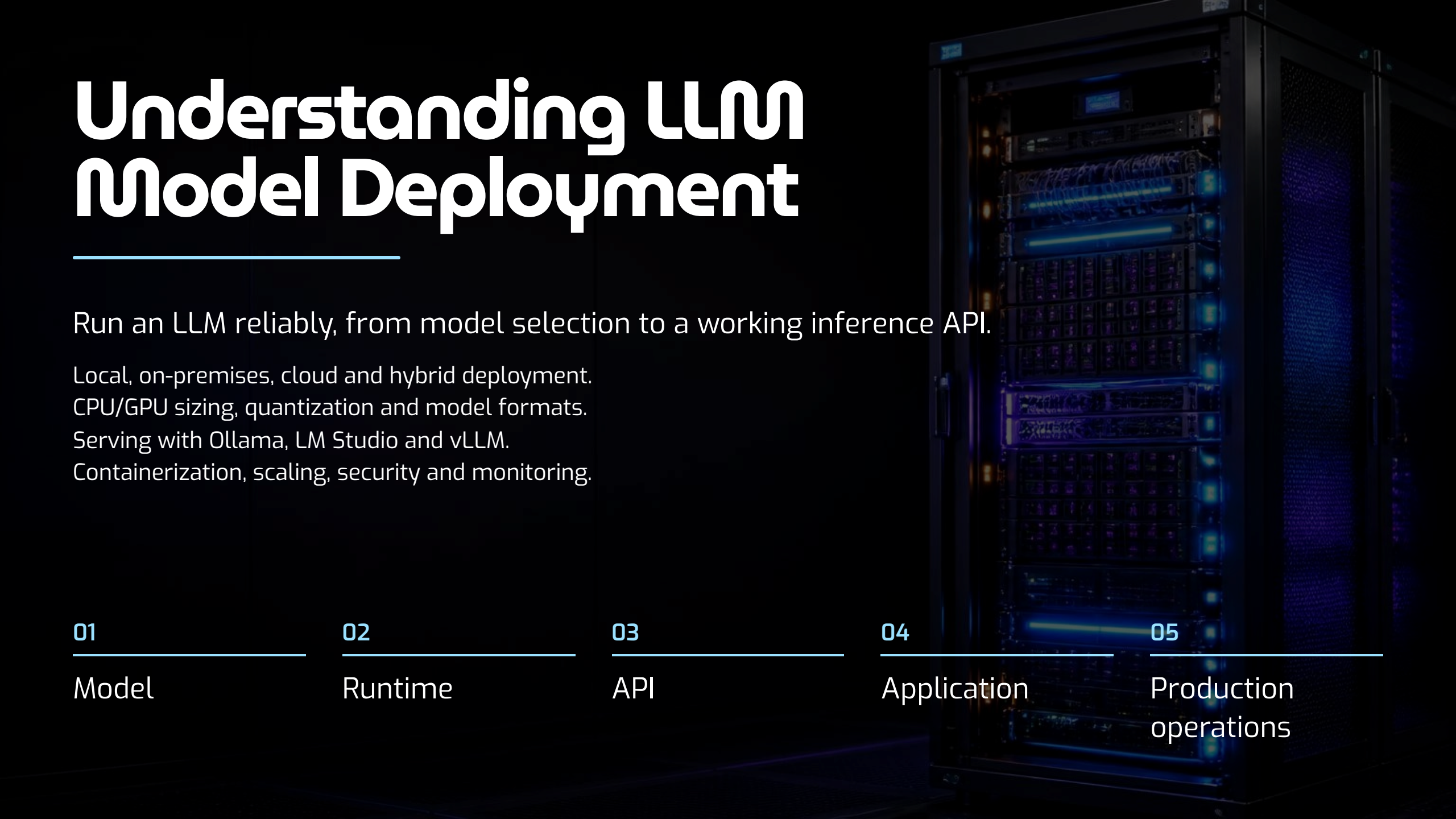
Understanding LLM Model Deployment

Run an LLM reliably, from model selection to a working inference API.

Local, on-premises, cloud and hybrid deployment.
CPU/GPU sizing, quantization and model formats.
Serving with Ollama, LM Studio and vLLM.
Containerization, scaling, security and monitoring.



01 Model

02 Runtime

03 API

04 Application

05 Production operations

LLM fundamentals

LLM: model that processes and generates language

Parameters: learned numerical values that determine behavior

Weights: learned tensors stored in model files

Tokenizer: converts text to tokens and back

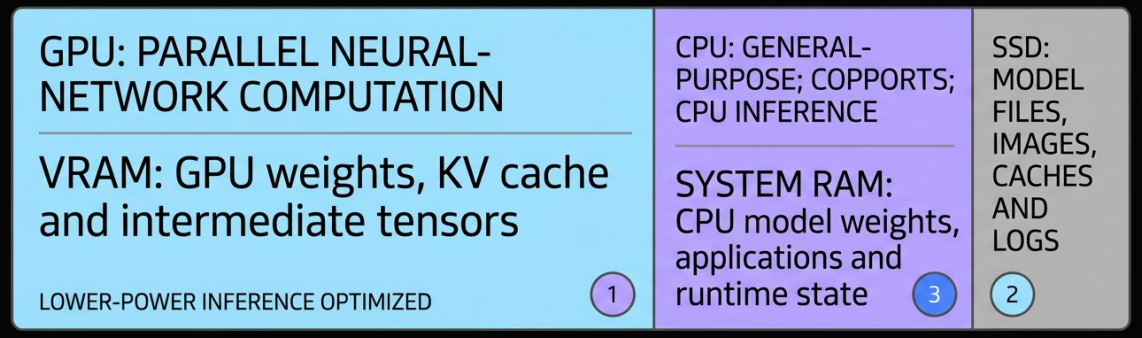
Context window: maximum context, measured in tokens

Inference: using the trained model to generate output

Flow

Text → | Tokens → | Transformer → | Generated tokens → | Text

Hardware: CPU, RAM, GPU and VRAM



Disk

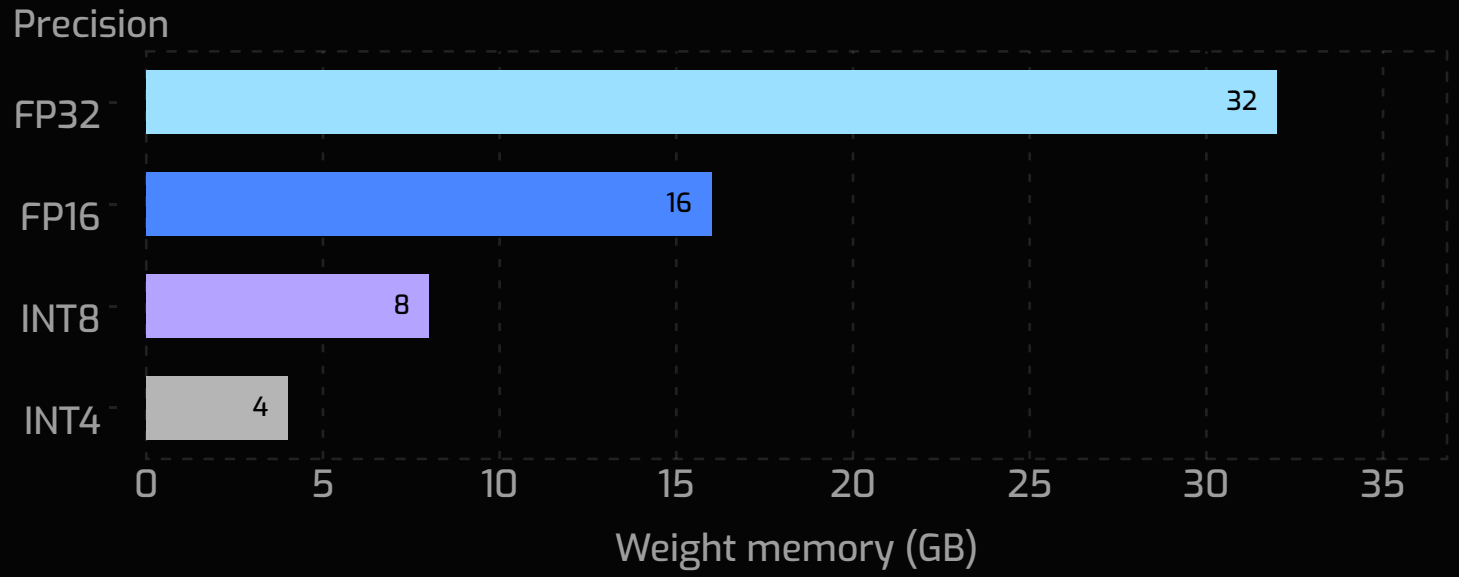
Model files, images, caches and logs.

Network

Downloads, API traffic and distributed inference.

Model size sets the memory floor, not the speed

8B MODEL · ESTIMATED WEIGHT MEMORY ONLY



$$\text{Weight memory} \approx \frac{\text{parameters} \times \text{bits per weight}}{8}$$

8B

About 16 GB for FP16 weights alone; about 4 GB at 4-bit.

Add KV cache, runtime overhead and temporary allocations.

Size alone does not determine latency or throughput.

PARAMETER COUNTS
0.5B · 3B · 8B · 70B

Quantization and optimization trade quality for footprint

01 Full-precision weights

02 Quantization

03 Smaller footprint

04 Quality evaluation

Precision

FP32, FP16, BF16, FP8

Quantization

INT8, INT4 and supported low-bit formats

Methods

GPTQ, AWQ, bitsandbytes

GGUF variants

Q4_K_M, Q5_K_M, Q8_0

Other optimization approaches include pruning, distillation, LoRA and QLoRA.

Memory and speed versus output quality.

Model formats: GGUF, Safetensors and ONNX

GGUF

Widely used by llama.cpp-based runtimes.

Safetensors

Common in Hugging Face repositories.

ONNX

Portable representation for supported runtimes.

PyTorch checkpoints

Common serialization files.

Sharded checkpoints

Large weights split across files.



VERIFY ARCHITECTURE, FORMAT, QUANTIZATION AND ENGINE COMPATIBILITY

How inference works

01

Prompt

02

Prefill

Processes the input tokens.

03

Decode

Decoding generates output tokens.

KV cache reuses attention state across decoding steps.

04

Streamed response

TTFT

Time to first token.

Throughput

Tokens or requests per unit of time.

Continuous batching

Schedules active requests dynamically.

Pick the runtime for the workload: Ollama, LM Studio, vLLM

Ollama

Simple local model management and API access.

LOCAL DEVELOPMENT

LM Studio

GUI-driven experimentation and local API.

EXPERIMENTATION

vLLM

High-throughput serving with batching and memory management.

PRODUCTION-ORIENTED SERVING

llama.cpp

Lightweight ecosystem, commonly GGUF.

SGLang

Alternative framework for advanced workloads.

CHOOSE BY HARDWARE, FORMATS, WORKLOAD AND OPERATIONS

APIs, Python and conversation state



API access

- REST and OpenAI-compatible APIs
- Base URL, model ID, authentication and request payload
- Python OpenAI SDK and provider-specific SDKs

Conversation state

- Chat messages, system prompts and streaming responses
- Store conversation history outside stateless serving requests
- External storage and app logic for multi-replica sessions

Docker-based model serving

01

Model server

Dockerfile defines how an image is built.

02

Docker image

Packaged application and runtime filesystem.

03

Running container

Running instance of an image.

04

Exposed API

Port mappings expose model APIs.

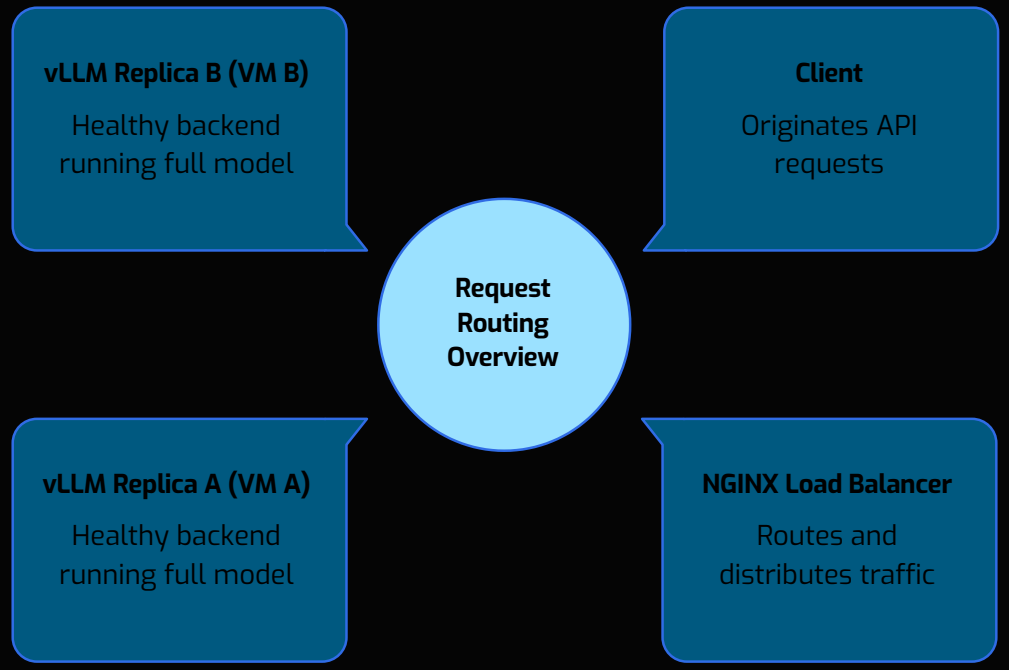
Persistence

Volumes persist model caches and configuration.

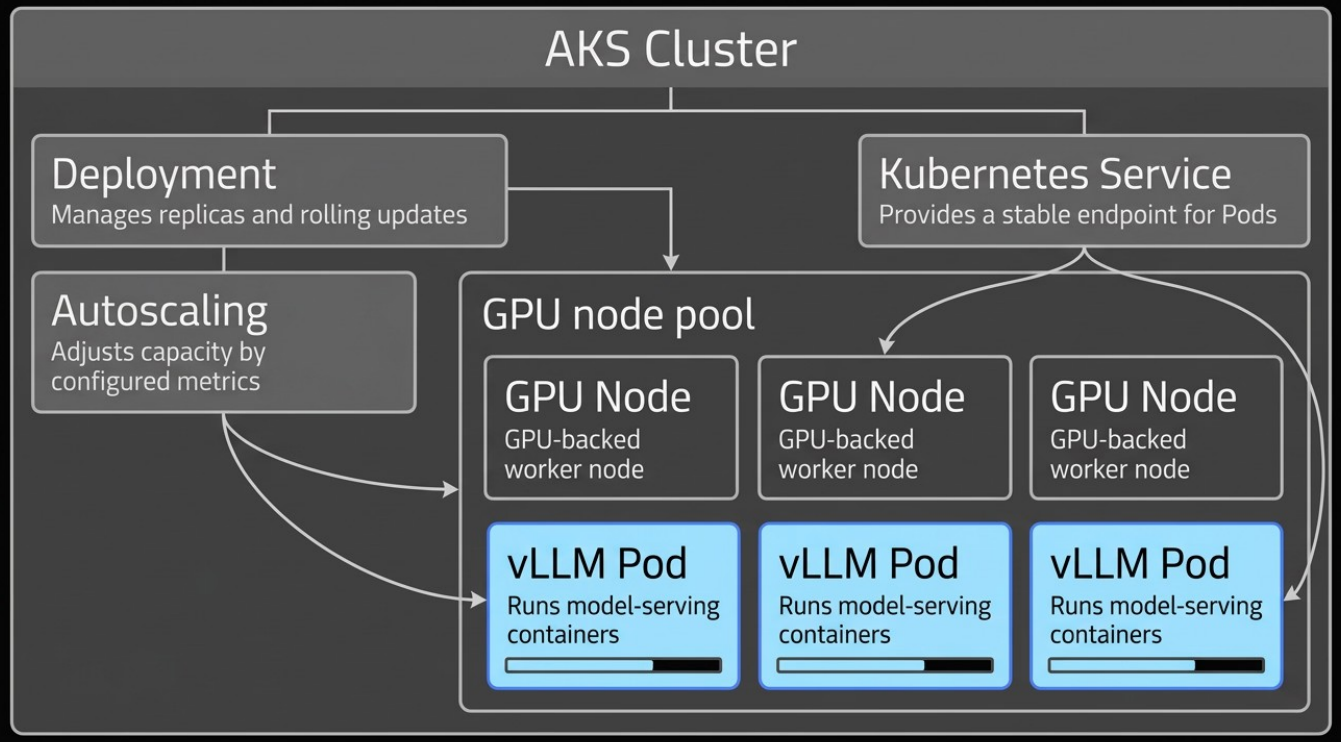
GPU support

GPU containers need compatible drivers and runtime support.

Multi-replica serving and load balancing



Kubernetes and AKS



Include secrets, monitoring, health checks and rollback.



On-premises, cloud and hybrid deployment

On-premises

More control; operations stay with the organization.

IaaS

Cloud VMs, storage and networks managed by the customer.

Managed AI

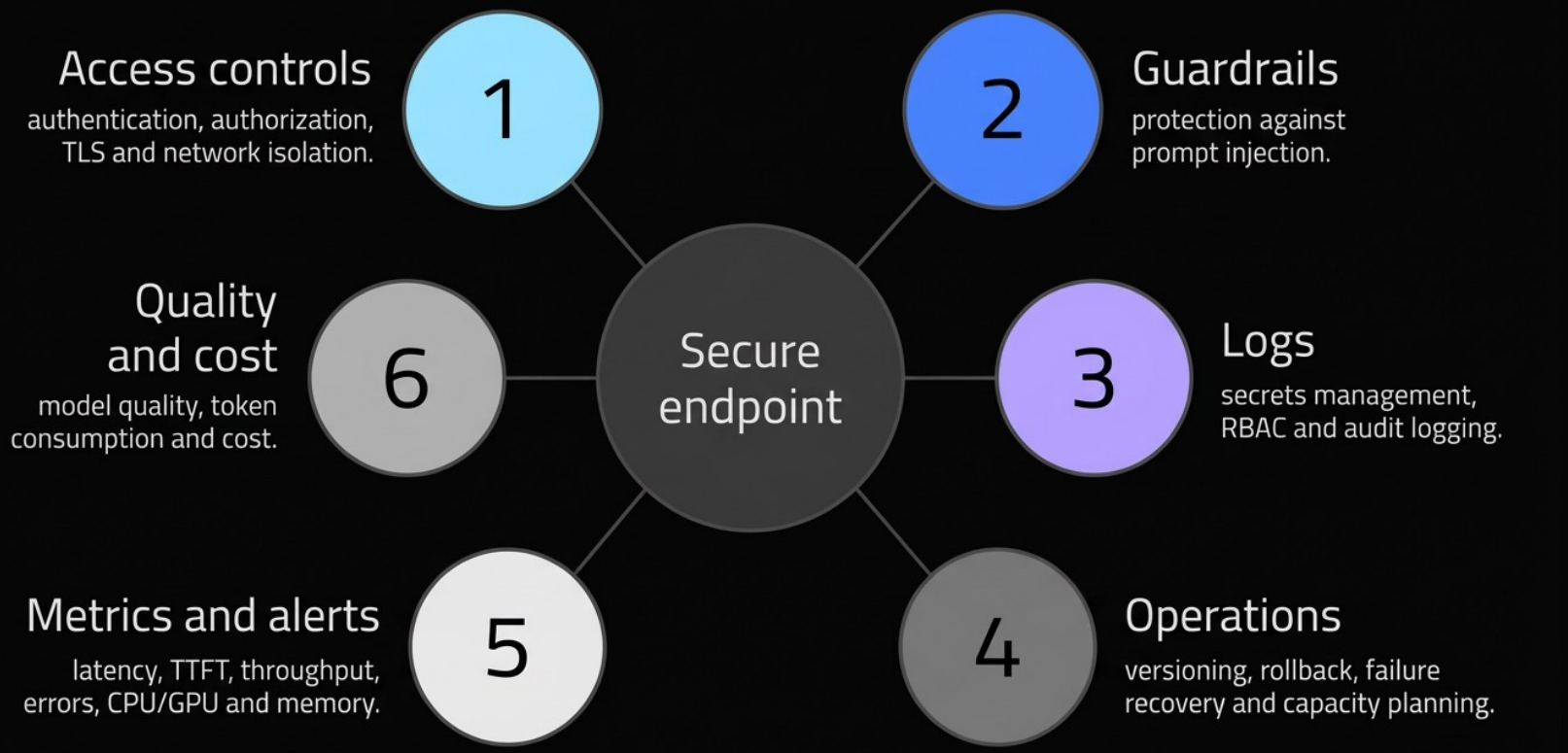
Provider operates much of the serving infrastructure.

Hybrid

Cloud training or development; on-premises inference where appropriate.



Production security and observability



Deployment decision framework: the Bosch use case

Standardize manufacturing incident reports.

01 Select a model; validate its license and capabilities.

02 Estimate memory; choose supported precision and format.

03 Select runtime, API architecture and compute.

04 Test factual accuracy, latency, concurrency and failure handling.

05 Deploy with monitoring, security, rollback and cost controls.

